# Programación de aplicaciones

Nombre: _____ Fecha: _____

## Qué vas a aprender

- Entender qué es una aplicación y cómo se organiza: interfaz y lógica
- Conocer el ciclo de desarrollo de una app
- Crear una aplicación sencilla para móvil con App Inventor
- Aplicar criterios de usabilidad, accesibilidad, privacidad y seguridad

## Para pensar

**Si usas decenas de apps cada día, ¿podrías crear una tú mismo en una clase?**

### Explicación

## 1.1 ¿Qué es una aplicación?

Una **aplicación** (*app*) es un programa diseñado para una tarea concreta: mensajería, juegos, mapas, banca... Según dónde funcionan, hay:

- **Apps nativas**: se instalan desde una tienda y se programan para un sistema concreto, **Android** o **iOS**. Aprovechan bien la cámara, el GPS o los sensores.
- **Apps web**: funcionan en el navegador, sin instalar, en cualquier dispositivo.
- **Apps de escritorio**: para ordenadores (procesadores de texto, editores).

Toda app tiene dos partes: la **interfaz** (lo que ves y tocas) y la **lógica** (el programa que decide qué pasa). Detrás, muchas usan además un **servidor** o una base de datos en la nube.

Toda app tiene dos partes: la **interfaz**, lo que el usuario ve y toca (pantallas, botones, textos, imágenes), y la **lógica**, el programa que decide qué ocurre en cada momento. Muchas apps tienen además una tercera parte invisible, el **servidor** o «backend», donde se guardan los datos compartidos entre usuarios, como los mensajes de un chat o las publicaciones de una red social.




---

## 1.2 El ciclo de desarrollo

Crear una app es un proyecto con fases, que se repiten varias veces hasta que funciona bien:

1. **Idea y análisis**: ¿qué problema resuelve?, ¿quién la usará?
2. **Diseño**: bocetos de cada pantalla (*wireframes*) y flujo de navegación.
3. **Programación**: interfaz y lógica.
4. **Pruebas**: en el simulador y en móviles reales, con usuarios.

5. **Publicación**: en una tienda de aplicaciones o en la web.
6. **Mantenimiento**: corregir errores y añadir mejoras (las actualizaciones).

Las empresas trabajan con **metodologías ágiles**: versiones pequeñas que se prueban y mejoran cada pocas semanas.

Este ciclo es **iterativo**: en lugar de intentar hacer la app perfecta de una vez, se crea primero una versión mínima que funcione (el **producto mínimo viable**), se prueba con usuarios reales y se mejora en sucesivas vueltas. Así se descubren pronto los errores de planteamiento y no se pierde tiempo programando funciones que nadie necesita.




### 1.3 Diseñar la interfaz

La **interfaz** es todo lo que el usuario ve y toca. Antes de programar se dibujan los **wireframes**: bocetos sencillos, en blanco y negro, de cada pantalla, con la posición de botones, textos e imágenes. Después se prepara un **prototipo** navegable, aunque no funcione de verdad, para comprobar si el recorrido es lógico.

Algunas reglas de diseño: usar una **paleta** de dos o tres colores coherente con el tema de la app; una o dos tipografías legibles, con tamaño suficiente; agrupar los elementos relacionados y alinearlos; dejar **espacio en blanco** para que la pantalla respire; colocar el botón principal donde el pulgar llega fácilmente (la parte inferior en los móviles); y mantener la **coherencia**: el mismo icono o color significa siempre lo mismo en todas las pantallas. En App Inventor, la disposición se controla con componentes de organización (**DisposiciónHorizontal**, **DisposiciónVertical** y **DisposiciónTabular**) y con las propiedades de ancho y alto, que pueden ajustarse al contenedor o expresarse en porcentaje de la pantalla.




## 2.1 App Inventor

**MIT App Inventor** es un entorno web gratuito para crear apps de Android (y algunas de iOS) con **programación por bloques**, como Scratch. Tiene dos vistas:

- **Diseñador**: arrastras **componentes** a la pantalla del móvil y ajustas sus **propiedades** (texto, color, tamaño).
- **Bloques**: programas qué ocurre cuando el usuario interactúa con esos componentes.

Para probar la app al momento se usa la aplicación **MIT AI2 Companion** en el móvil, que se conecta escaneando un código QR. Al terminar, se genera un archivo `.apk` para instalarla.

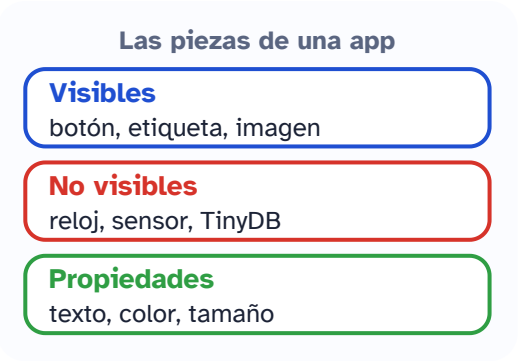



## 2.2 Componentes y propiedades

| Componente | Para qué sirve | Propiedades típicas |
|---|---|---|
| **Botón** | que el usuario toque | Texto, ColorDeFondo, Imagen |
| **Etiqueta** | mostrar texto | Texto, TamañoDeLetra |
| **Caja de texto** | que el usuario escriba | Pista, SoloNúmeros |

| Componente | Para qué sirve | Propiedades típicas |
|---|---|---|
| **Imagen** | mostrar una foto o dibujo | Foto, Ancho, Alto |
| **Disposiciones** | ordenar en filas o columnas | Alineación |
| **No visibles** | TextoAVoz, Sonido, Acelerómetro, Cámara, Reloj, TinyDB | según el componente |

Los componentes **no visibles** no aparecen en pantalla pero dan acceso a las funciones del móvil: hablar, vibrar, medir la inclinación, guardar datos…




## 2.3 Programar con eventos

Las apps funcionan por **eventos**: no hacen nada hasta que pasa algo (el usuario toca un botón, agita el móvil, llega un mensaje). Cada evento tiene su bloque:

```
cuando BotónHablar.Clic
    llamar TextoAVoz1.Hablar
        mensaje: CajaTexto1.Texto
```

Este programa lee en voz alta lo que el usuario haya escrito. Otros eventos útiles: **Pantalla1.Inicializar** (al abrir la app), **Acelerómetro.Agitado**, **Reloj.Temporizador** (cada cierto tiempo). Dentro de los eventos se usan variables, condicionales y bucles, exactamente igual que en Scratch.

Además de los botones, hay muchos otros eventos: **Pantalla1.Inicializar** (al abrir la app: buen momento para cargar datos guardados), **CuadroDeTexto.PerdióFoco** (al terminar de escribir), **Acelerómetro.Agitar**, **Reloj.Temporizador** (se repite cada cierto tiempo, útil para cronómetros y juegos) o **Lienzo.Tocado** (al tocar en una zona de dibujo). Una app bien hecha responde a los eventos del usuario sin quedarse bloqueada.




## 2.4 Variables, listas y procedimientos

Como en Scratch, en App Inventor se crean **variables globales** (con «inicializar global nombre como») que existen mientras la app está abierta, y **variables locales**, que solo existen dentro de un bloque. Las **listas** guardan muchos elementos: la lista de tareas, las preguntas de un cuestionario o los turnos de riego. Hay bloques para añadir, quitar, seleccionar un elemento por su índice (en App Inventor el primero es el 1) y recorrer la lista con «para cada elemento en la lista».

Para conectar las listas con la pantalla se usa el componente **VisorDeLista**: basta con asignar la lista a su propiedad Elementos para que se muestre, y el evento DespuésDeSelección indica qué elemento ha tocado el usuario. Cuando un conjunto de bloques se repite, se agrupa en un **procedimiento** («como procedimiento actualizarLista»), que se llama desde varios eventos. Si devuelve un valor, como el total de puntos, se usa «procedimiento con resultado». Así el programa es más corto y fácil de mantener.

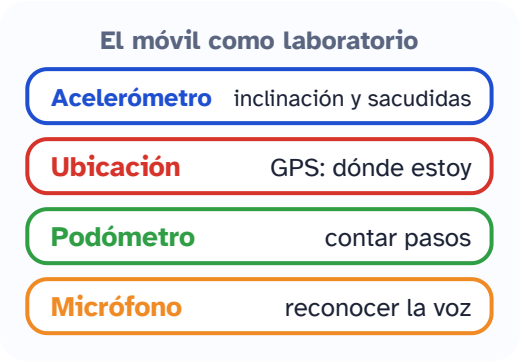



## 2.5 Usar los sensores del móvil

Un móvil es un pequeño laboratorio lleno de sensores, y App Inventor da acceso a muchos de ellos como componentes no visibles. El **acelerómetro** mide la

inclinación y detecta sacudidas: sirve para un dado que se lanza al agitar o un juego en el que una bola rueda al inclinar el móvil. El **sensor de ubicación** (GPS) da la latitud, la longitud y la velocidad: una app de rutas o de deporte. El **sensor de orientación** funciona como una brújula. El **podómetro** cuenta pasos. Y el **sensor de luz** o de **proximidad** detectan la luz ambiental o si algo está cerca de la pantalla.

También se pueden usar la **cámara**, el **micrófono** (reconocimiento de voz para dictar), el altavoz (TextoAVoz) y la **vibración**. Cada sensor tiene sus eventos (por ejemplo, Acelerómetro.CambioDeAceleración) y sus propiedades (Habilitado, IntervaloDeTiempo). Conviene desactivar los sensores cuando no se usan, porque gastan batería, y pedir permiso al usuario para los que afectan a su privacidad, como la ubicación o el micrófono.




## 2.6 Guardar datos y usar varias pantallas

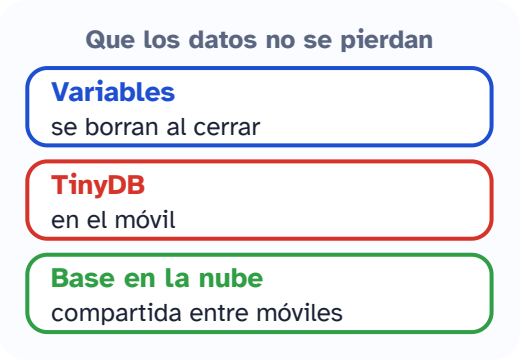
Si cierras una app, sus variables se borran. Para que los datos **persistan** (una lista de tareas, la mejor puntuación) se guardan en una base de datos:

- **TinyDB**: guarda datos en el propio móvil con una **etiqueta** y un **valor**.
- **Bases de datos en la nube**: comparten datos entre varios usuarios, como en un chat.

Las apps más completas tienen **varias pantallas** (inicio, juego, ajustes). Se pasa de una a otra con el bloque «abrir otra pantalla». Antes de programarlas, dibuja el **mapa de navegación**: qué pantallas hay y qué botón lleva a cada una.

Para pasar de una pantalla a otra se usa el bloque «abrir otra pantalla nombreDePantalla», y para volver, «cerrar pantalla». Cada pantalla tiene sus propios componentes y bloques, y las variables no se comparten entre pantallas:

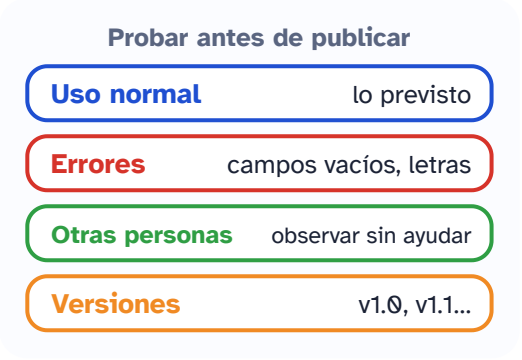
si hace falta pasar un dato, se envía como valor inicial al abrir la nueva pantalla o se guarda en TinyDB.




## 3.1 Probar la app y corregir errores

Una app nunca está terminada sin **pruebas**. Mientras se programa, App Inventor permite probarla en directo en el móvil con la aplicación **Companion** (escaneando un código QR) o en un emulador: cada cambio se ve al instante. Pero hay que probar también instalando el archivo .apk, en distintos móviles y tamaños de pantalla.

Un buen plan de pruebas incluye una **lista de casos**: qué se hace, qué debería pasar y qué pasa de verdad. Hay que probar el uso normal, pero también lo que un usuario haría por error: pulsar un botón dos veces seguidas, dejar un campo vacío, escribir letras donde se esperan números, girar el móvil o cerrar la app a mitad. Muy importante: que la prueben **otras personas**, sin explicarles nada, observando dónde dudan. Los errores se anotan, se corrigen de uno en uno y se vuelve a probar. Cada versión estable se guarda con un número (v1.0, v1.1…) usando «Guardar proyecto como», para poder volver atrás si algo se estropea.




## 3.2 Usabilidad y accesibilidad

Una app técnicamente perfecta fracasa si es difícil de usar. La **usabilidad** mide lo fácil que resulta:

- Botones **grandes** y separados (al menos el tamaño de la yema de un dedo).
- Textos **claros** y breves; iconos conocidos.
- **Coherencia**: los mismos colores y posiciones en todas las pantallas.
- **Respuesta**: cuando el usuario toca algo, debe notar que ha pasado algo.

La **accesibilidad** busca que todo el mundo pueda usarla: buen contraste, letra ampliable, descripciones de las imágenes para los lectores de pantalla y no depender solo del color para dar información.

La accesibilidad no es un extra: en Europa existen normas que obligan a muchas apps y webs públicas a ser accesibles. Algunas pautas sencillas: un contraste alto entre texto y fondo, no transmitir información solo con el color (añadir iconos o texto), descripciones de las imágenes para los lectores de pantalla, permitir agrandar la letra y ofrecer alternativas por voz.



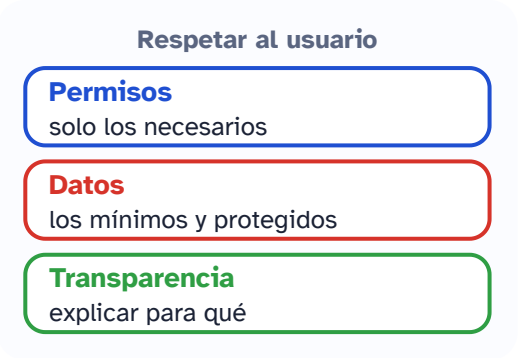

### 3.3 Privacidad, seguridad y ética

Las apps pueden pedir **permisos**: cámara, micrófono, ubicación, contactos. Como usuario, concede solo los imprescindibles. Como programador, pide solo los que necesites y explica para qué.

- Protege los datos personales de los usuarios; en Europa lo exige el **Reglamento General de Protección de Datos** (RGPD).
- Descarga apps solo de tiendas oficiales.
- Respeta los **derechos de autor** de imágenes y sonidos que uses.
- Evita diseños que enganchen o manipulen, como notificaciones constantes.

Una buena app resuelve un problema real **sin aprovecharse** de quien la usa.

Un principio útil es la **privacidad desde el diseño**: pensar en proteger los datos desde la primera idea de la app, no al final. Eso supone recoger solo los datos imprescindibles, no pedir el nombre completo si basta un apodo y borrar lo que ya no hace falta.




## 3.4 Publicar y mantener una app

Desde App Inventor se genera un archivo **.apk** que se puede compartir e instalar en móviles Android (hay que permitir la instalación de orígenes desconocidos) o un código QR para descargarlo. Para publicar en las tiendas oficiales, como Google Play, se necesita una cuenta de desarrollador, una ficha con el nombre, la descripción, capturas de pantalla, el icono, la clasificación por edades y una **política de privacidad** que explique qué datos se recogen y para qué. Las tiendas revisan las apps antes de publicarlas.

Publicar no es el final. Una app viva necesita **mantenimiento**: corregir los errores que comunican los usuarios, adaptarla a nuevas versiones del sistema operativo, añadir mejoras y publicar **actualizaciones** con su número de versión y una nota de lo que ha cambiado. Las valoraciones y comentarios de los usuarios son una fuente de ideas muy valiosa. Muchas apps de éxito empezaron siendo pequeñas y mejoraron versión a versión escuchando a quienes las usaban.



### 3.5 Otras herramientas y lenguajes

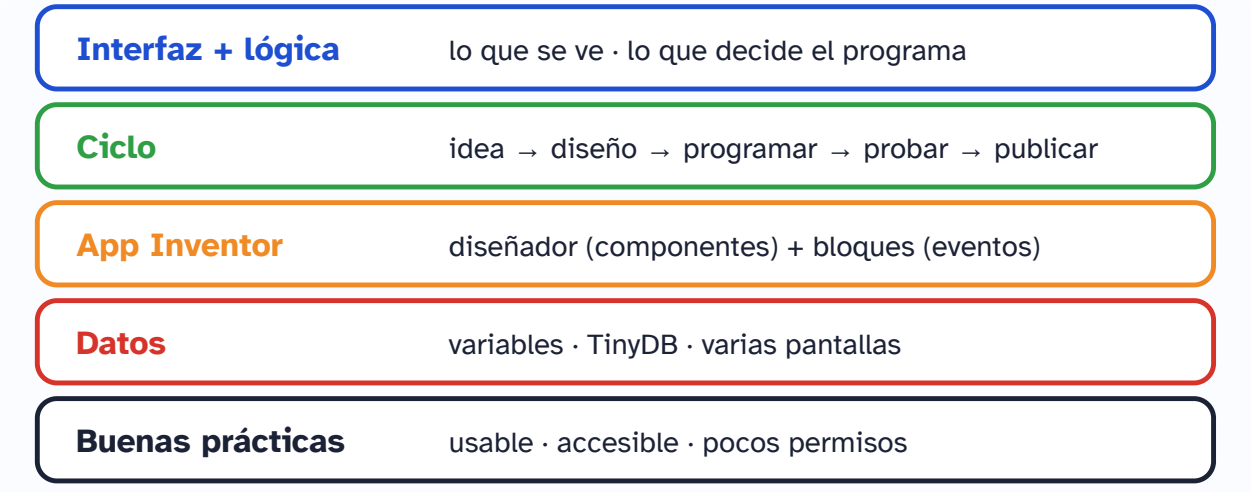
App Inventor es ideal para aprender. Otras opciones:

- **Thunkable** y **Kodular**: también por bloques, con más componentes.
- **Code.org App Lab**: apps web con bloques o JavaScript.
- Profesionales: **Android Studio** (Kotlin o Java) para Android, **Xcode** (Swift) para iOS, o **Flutter** y React Native para ambas a la vez.

La lógica que has aprendido (eventos, variables, condicionales, bucles, listas) es la misma en todas. Lo que cambia es la forma de escribirla.




## Crear una app



## Ejercicios

1. **Explica la diferencia entre una app nativa y una app web.**

2. **Ordena las fases: pruebas, idea, publicación, programación, diseño.**

3. **¿Qué componentes necesitas para una app que convierta euros a dólares?**

4. **Escribe el bloque para la app anterior con un cambio de 1,08.**

5. **¿Qué componente no visible usarías para una app que cuente pasos? ¿Y para una linterna?**

6. **Nombra cuatro criterios de usabilidad.**

7. **Nombra tres medidas de accesibilidad.**

8. **Una app de linterna pide acceso a tus contactos y ubicación. ¿Qué harías?**

9. **Dibuja el mapa de navegación de una app de recetas con tres pantallas.**

10. **Diseña y programa una app de lista de tareas que guarde los datos.**

11. **¿Qué es un wireframe y para qué sirve?**

12. **Nombra tres componentes de organización de App Inventor.**

13. **¿Qué sensor usarías para un dado que se lanza al agitar? ¿Y para una app de rutas?**

14. **¿Qué es Companion?**

15. **¿Qué debe explicar la política de privacidad de una app?**

16. **Escribe tres reglas de diseño de interfaces.**

17. **¿Qué diferencia hay entre un procedimiento y un procedimiento con resultado?**

18. **Escribe cinco casos de prueba para una app que convierte euros a dólares.**

19. **Programa un cronómetro con el componente Reloj.**

20. **¿Por qué conviene desactivar los sensores cuando no se usan?**

21. **Diseña la lógica para eliminar de la lista la tarea que el usuario toque.**

22. **Explica por qué es importante que la app la prueben personas que no la han programado.**

23. **¿Qué hace falta para publicar una app en Google Play?**

24. **Programa un contador de pasos que muestre los pasos y los metros (0,7 m por paso).**

25. **Propón tres mejoras de accesibilidad para una app de horarios de autobús.**

26. **Diseña completa la app del huerto de la lectura: pantallas, componentes, datos guardados y eventos principales.**

27. **Una app de deporte pide ubicación, contactos, cámara y micrófono. Analiza qué permisos son razonables y cuáles no.**

28. **Haz un juego de «atrapa el topo» con un Lienzo, un SpriteImagen y un Reloj, con puntos y tiempo límite.**

29. **Explica qué ventajas tendría guardar los datos de la app del huerto en la nube en lugar de en TinyDB, y qué riesgos.**

30. **Redacta la nota de actualización de la versión 1.1 de una app de tareas que corrige un error y añade una función.**

## Practica

1. **Las dos partes de toda app son...**

   **A)** pantalla y batería
   **B)** interfaz y lógica
   **C)** icono y nombre
   **D)** sonido y vídeo

2. **Una app que funciona en el navegador sin instalar es una app...**

   **A)** nativa
   **B)** de escritorio
   **C)** web
   **D)** de pago

3. **App Inventor fue creado por el...**

   **A)** MIT
   **B)** BBC
   **C)** Apple
   **D)** Google

4. **En App Inventor, los botones y etiquetas se colocan en el...**

   **A)** editor de bloques
   **B)** Companion
   **C)** Diseñador
   **D)** Companion

5. **El componente que lee un texto en voz alta es...**

   **A)** Etiqueta
   **B)** TextoAVoz
   **C)** TinyDB
   **D)** Sonido

6. **Para guardar datos en el móvil aunque se cierre la app se usa...**

   **A)** TinyDB
   **B)** una variable global
   **C)** una etiqueta
   **D)** un reloj

7. **Las apps funcionan principalmente por...**

**A)** orden alfabético
**B)** eventos
**C)** azar
**D)** los permisos

8. **Que una app la pueda usar una persona con baja visión es...**

**A)** rendimiento
**B)** compilación
**C)** accesibilidad
**D)** seguridad

9. **El boceto de una pantalla se llama...**

**A)** wireframe
**B)** apk
**C)** sprite
**D)** icono

10. **Como programador, debes pedir...**

**A)** todos los permisos
**B)** solo los permisos necesarios
**C)** ningún dato nunca
**D)** ningún permiso nunca, aunque sean necesarios

11. **El boceto sencillo de una pantalla es un...**

**A)** sprite
**B)** wireframe
**C)** apk
**D)** permiso

12. **En App Inventor, para colocar botones en fila se usa...**

**A)** DisposiciónHorizontal
**B)** TinyDB
**C)** Reloj
**D)** Notificador

13. **Para mostrar una lista en pantalla se usa el...**

**A)** VisorDeLista
**B)** Acelerómetro
**C)** TextoAVoz

**D)** Lienzo

14. **En App Inventor, el primer elemento de una lista tiene índice…**

**A)** 0
**B)** 1
**C)** –1
**D)** 10

15. **Un conjunto de bloques reutilizable es un…**

**A)** evento
**B)** procedimiento
**C)** permiso
**D)** sensor

16. **Para detectar que se agita el móvil se usa el…**

**A)** GPS
**B)** acelerómetro
**C)** podómetro
**D)** micrófono

17. **Companion sirve para…**

**A)** publicar en la tienda
**B)** probar en directo en el móvil
**C)** dibujar iconos
**D)** cobrar

18. **Un buen caso de prueba incluye…**

**A)** solo el uso normal
**B)** también errores del usuario
**C)** solo el diseño
**D)** nada

19. **La política de privacidad explica…**

**A)** el precio
**B)** qué datos recoge la app y para qué
**C)** el color
**D)** la versión

20. **Las actualizaciones de una app sirven para…**

**A)** borrar datos

**B)** corregir errores y mejorar

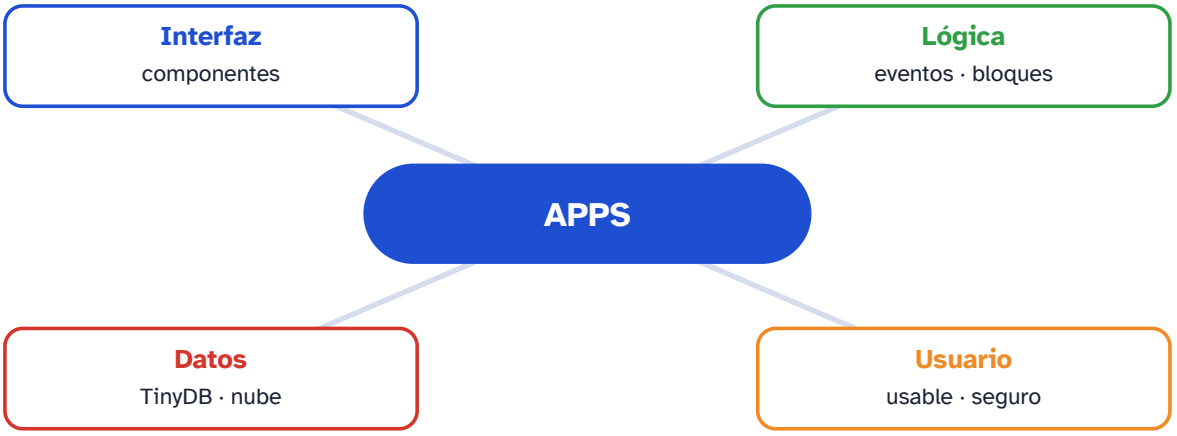
**C)** pedir más permisos

**D)** nada

## Explícalo con tus palabras

Incluye: Qué diferencia hay entre interfaz y lógica; Las fases del desarrollo de una app; Cómo se programa un botón en App Inventor; Qué hace usable y accesible una aplicación.

## Resumen




## Glosario

| | |
|---|---|
| **Aplicación** | Programa diseñado para una tarea concreta. |
| **App nativa** | Se instala y se programa para un sistema. |
| **App web** | Funciona en el navegador. |
| **Interfaz** | Lo que el usuario ve y toca. |
| **Wireframe** | Boceto de una pantalla. |
| **Componente** | Pieza de una app: botón, etiqueta, sensor. |
| **Propiedad** | Característica de un componente. |
| **Evento** | Suceso que dispara unos bloques. |
| **Procedimiento** | Bloques agrupados con un nombre. |
| **VisorDeLista** | Componente que muestra listas. |
| **Acelerómetro** | Sensor de aceleración e inclinación. |
| **TinyDB** | Base de datos en el propio móvil. |
| **Companion** | App para probar en directo. |
| **APK** | Archivo de instalación de Android. |
| **Usabilidad** | Facilidad de uso. |
| **Accesibilidad** | Que todas las personas puedan usarla. |
| **Permiso** | Autorización para acceder a datos o sensores. |

## Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

**App nativa**

Se instala y se programa para Android o iOS.

**Interfaz**

La parte de la app que se ve y se toca.

**Componente**

Elemento de la app: botón, etiqueta, sensor...

**Propiedad**

Característica de un componente: texto, color, tamaño.

**Evento**

Algo que ocurre y dispara un programa: un clic, agitar.

**TinyDB**

Base de datos en el móvil para guardar datos.

**Usabilidad**

Facilidad de uso de una aplicación.

**Permiso**

Acceso que pide una app a cámara, ubicación, etc.

**Wireframe**

Boceto de una pantalla antes de programar.

**VisorDeLista**

Componente que muestra una lista en pantalla.

**Acelerómetro**

Sensor de inclinación y sacudidas.

**Política de privacidad**

Explica qué datos recoge una app y para qué.

## Soluciones

1. **B)** Lo que se ve y lo que decide.
2. **C)** Cualquier dispositivo.
3. **A)** Como Scratch.
4. **C)** Vista de diseño.
5. **B)** No visible.
6. **A)** Etiqueta + valor.
7. **B)** Responden a lo que pasa.
8. **C)** Para todo el mundo.
9. **A)** Esquema de diseño.
10. **B)** Y explicar para qué.
11. **B)** Se dibuja antes de programar.
12. **A)** Componente de organización.
13. **A)** Propiedad Elementos.
14. **B)** Empieza en 1.
15. **B)** Se llama desde varios sitios.
16. **B)** Mide la aceleración.
17. **B)** Escaneando un QR.
18. **B)** Campos vacíos, letras...
19. **B)** Obligatoria en las tiendas.
20. **B)** Mantenimiento.

## Soluciones de los ejercicios

1. La nativa se instala y se programa para un sistema; la web funciona en el navegador sin instalar.
2. Idea, diseño, programación, pruebas, publicación (y mantenimiento).
3. Caja de texto (cantidad), botón (convertir) y etiqueta (resultado).
4. cuando Botón.Clic → poner Etiqueta.Texto a CajaTexto.Texto · 1,08.
5. Podómetro (o acelerómetro); para la linterna, el componente de la cámara con flash o la pantalla en blanco.
6. Botones grandes, textos claros, coherencia visual, respuesta a las acciones.
7. Buen contraste, letra ampliable, descripción de imágenes para lectores de pantalla.
8. Desconfiar y no instalarla o denegar esos permisos: no los necesita.
9. Inicio → lista de recetas → detalle de la receta, con botón de volver.
10. Respuesta abierta; caja de texto, botón añadir, vista de lista y TinyDB para guardar la lista.
11. Un boceto sencillo de una pantalla con la posición de sus elementos; sirve para pensar el diseño antes de programar.
12. DisposiciónHorizontal, DisposiciónVertical y DisposiciónTabular.
13. Acelerómetro; sensor de ubicación (GPS).
14. La app de App Inventor que permite probar el proyecto en directo en el móvil, escaneando un código QR.
15. Qué datos recoge, para qué los usa, con quién los comparte y cómo puede el usuario borrarlos.
16. Paleta de pocos colores coherentes, letra legible y suficientemente grande, botón principal al alcance del pulgar (y coherencia entre pantallas).
17. El procedimiento ejecuta bloques; el procedimiento con resultado, además, devuelve un valor que se puede usar (por ejemplo, el total calculado).
18. Número normal (10 → 10,80); cero; campo vacío; letras en vez de número; número con coma decimal. Comprobar en cada caso qué muestra y que no se cierre.
19. Variable global segundos = 0; Reloj con IntervaloDeTiempo 1000 ms. Cuando Reloj.Temporizador: segundos ← segundos + 1; Etiqueta.Texto ← segundos. Botones para habilitar y deshabilitar el reloj.
20. Porque consumen batería y, en el caso de la ubicación o el micrófono, pueden recoger datos sin necesidad.
21. cuando VisorDeLista.DespuésDeSelección: quitar elemento de lista tareas en el índice VisorDeLista.ÍndiceSeleccionado; fijar VisorDeLista.Elementos a tareas; guardar en TinyDB.
22. Porque quien la programa sabe cómo funciona y no comete los errores de un usuario nuevo; observar a otros revela botones confusos, pasos que no se entienden y fallos inesperados.
23. Cuenta de desarrollador, archivo de la app, ficha (nombre, descripción, icono, capturas), clasificación por edades y política de privacidad.

24. Componente Podómetro. cuando Podómetro.PasoDetectado: EtiquetaPasos.Texto ← Podómetro.Pasos; EtiquetaMetros.Texto ← Podómetro.Pasos · 0,7.
25. Letra ampliable y alto contraste, botón que lee en voz alta el próximo autobús y no indicar los retrasos solo con color rojo, sino también con texto.
26. Pantallas: Inicio, Turnos, Historial. Componentes: botones, VisorDeLista, TextoAVoz, TinyDB (o base en la nube para compartir), Notificador. Datos: lista de turnos y lista de riegos (fecha y nombre). Eventos: Pantalla.Inicializar carga los datos; BotónRegado.Clic añade un riego con la fecha y guarda; BotónConsejos.Clic lee el consejo de la planta en voz alta.
27. Ubicación: razonable para registrar rutas (mejor solo mientras se usa). Cámara: aceptable si permite subir fotos de la ruta. Contactos y micrófono: injustificados para registrar ejercicio; denegarlos y desconfiar de la app.
28. Reloj cada 800 ms: mover el topo a una x e y aleatorias. cuando Topo.Tocado: puntos ← puntos + 1; vibrar 100 ms. Otro reloj de 1 s: tiempo ← tiempo – 1; si tiempo = 0: deshabilitar relojes y mostrar puntos con el Notificador.
29. Ventaja: todos los móviles comparten los mismos turnos e historial al momento. Riesgos: hace falta conexión, los datos están en un servidor externo y hay que protegerlos (acceso con contraseña, no guardar datos personales innecesarios) y cumplir el RGPD.
30. Versión 1.1: «Corregido: las tareas ya no se duplican al pulsar dos veces el botón Añadir. Nuevo: ahora puedes marcar tareas como completadas tocándolas, y se muestran tachadas. Gracias por vuestros comentarios.»